[原创]ARM64 OLLVM反混淆 精

无名侠 5 10

学者

2019-6-29 16:19 17122

基于Unicorn 的ARM64 OLLVM反混淆

0x0

最近在研究娜迦加固，娜迦加固的SO文件被ollvm混淆，转而研究ollvm反混淆。网上的很多关于ollvm反混淆的资料都基于符号执行。符号执行主要用框架是Miasm和Angr。Miasm和Angr对arm64指令集支持都不是很好，于是采用模拟执行的方式来寻找两个真实块的关系。**本文仅提供大体思路，如果有更好的方法欢迎一起讨论，细节优化还要根据具体情况分析**

我把ollvm混淆后的代码块分为两类：虚假块和真实块。虚假块仅包含ollvm流程控制指令，不含任何真实指令。只要含有真实指令的代码块就是真实块。

反ollvm混淆的关键是找出两个真实块之间的关系。

两个真实块之间的关系有两种：1、顺序；2、分支

```
graph TD
A[真实块1] -->|A1| B(真实块2)
B --> C{csel 指令条件判断}
C -->|True| E[真实块3]
C -->|False| F[真实块4]
```

使用符号执行、模拟执行找出真实块之间的关系之后，patch相应的跳转即可实现反混淆。

0x1 反汇编、代码块、CFG分析

要获得目标函数的代码块和CFG，首先要反汇编。反汇编使用Capstone 反汇编引擎。

```
offset = 0x70438 #function start
end = 0x7170C    #function end

bin1 = open('libvdog.so','rb').read()
md = Cs(CS_ARCH_ARM64,CS_MODE_ARM)
md.detail = True #enable detail analyise

list_blocks = {}
block_item = {}
processors = {}
dead_loop = []
real_blocks = []
csel_list_cond = {}


isNew = True
insStr = ''
for i in md.disasm(bin1[offset:end],offset):
    insStr += "0x%x:\t%s\t%s\n" %(i.address, i.mnemonic, i.op_str)
    if isNew:
        isNew = False
        block_item = {}
        block_item["saddress"] = i.address
        block_item['capstone'] = []

    block_item['capstone'].append(i)
    block_item['flag'] = False


    if len(i.groups) > 0 or i.mnemonic == 'ret':
        isNew = True
        block_item["eaddress"] = i.address
        block_item['ins'] = insStr
        insStr = ''
        for op in i.operands:
            if op.type == ARM64_OP_IMM:
                block_item["naddress"] = op.value.imm

                if op.value.imm == i.address:
                    print "dead loop:%x" % i.address
                    dead_loop.append(i.address)

                if not processors.has_key(op.value.imm):
                    processors[op.value.imm] = 1
                else:
                    processors[op.value.imm] += 1
        if not block_item.has_key("naddress"):
            block_item['naddress'] = None
        list_blocks[block_item["saddress"]] = block_item

#delete dead loop
for dead in dead_loop:
    if processors.has_key(dead):
        del(processors[dead])
```

娜迦的样本中存在死循环路径，必须删掉。

0x2 识别真实块

真实块结束后会重新跳转入ollvm分发流程的代码块，该代码块一定有很多引用，所以反汇编代码块的时候记录后继节点的引用数目（见前一段代码）

```
if not processors.has_key(op.value.imm):
    processors[op.value.imm] = 1
else:
    processors[op.value.imm] += 1
```

反汇编和代码块识别完成后，如果某个代码块的引用大于1，就可能是分发器，引用该分发器的代码块就可能为真实块,筛选代码如下：

```
for b in list_blocks:
    if processors.has_key(list_blocks[b]['naddress']):
        if processors[list_blocks[b]['naddress']] > 1:
            real_blocks.append(list_blocks[b]['saddress'])
```

真实情况下，虚假代码块也会引用分发器，所以还要进一步筛选。

```
ssign = {u'b2',
 u'cmp11b.ne2',
 u'cmp11mov11b.ne2',
 u'movz12movk12b2',
 u'movz12movk12cmp11b.ne2',
 u'movz12movk12cmp11mov11b.ne2',
 'movz12movk12cmp11mov11movz12movk12movz12movk12b.ne2',
 u'movz12movk12cmp11movz12movk12b.eq2',
 'movz12movk12cmp11movz12movk12b.ne2',
 'movz12movk12movz12movk12b2',
 'movz12movk12cmp11movz12movk12movz12movk12movz12movk12b.eq2',
 'movz12movk12movz12movk12cmp11b.eq2',
 'movz12movk12movz12movk12cmp11movz12movk12b.eq2',
 'movz12movk12cmp11b.eq2',
         'ldr13b2',
         'mov11movz12movk12cmp11movz12movk12b.eq2'
 }
ssign2 = set()
def  is_real_blocks(ins):
    sign = get_code_sign(ins)
    if sign in ssign:
        return False
    if sign.endswith('movk12movz12movk12b.ne2'):
        return False

    for insn in item['capstone']:
        #print insn.mnemonic
        if insn.mnemonic not in ['movz','movk','cmp','b.eq','b.ne']:
            return True
    ssign2.add(sign)
    return False


fake_blocks = []
for i in real_blocks:
    item = list_blocks[i]
    if not is_real_blocks(item):
        print '## fake block ###'
        print item['ins']
        fake_blocks.append(i)
```

采用特征码的方法识别ollvm的虚假块。采用的特征码与寄存器、常量无关，只与指令类型、操作数类型有关。计算一个代码块的特征后进行匹配即可筛掉大量虚假代码块。

如果虚假块没有识别出来，那么可能导致后文的死循环，也是一种筛选思路。

按照定义，含有ret的代码块也属于真实块，但是前文真实块的识别基于引用 大量代码块的共同引用代码块，ret指令显然不会引用任何代码块，故单独处理。

```
for i in list_blocks:
    if list_blocks[i]['ins'].find('ret') != -1:
        print 'ret block:%x' % i
        real_blocks.append(i)
```

使用 Unicorn 模拟执行

初始化虚拟机

关于Unicorn的教程请自行Google，这里仅讨论Unicorn在寻找路径方面的应用。

```
mu = Uc(UC_ARCH_ARM64, UC_MODE_ARM)
#init stack
mu.mem_map(0x80000000,0x10000 * 8)

mu.mem_map(0, 4 * 1024 * 1024)
mu.mem_write(0,bin1)
mu.reg_write(UC_ARM64_REG_SP, 0x80000000 + 0x10000 * 6)
mu.hook_add(UC_HOOK_CODE, hook_code)
mu.hook_add(UC_HOOK_MEM_UNMAPPED,hook_mem_access)
```

Unicorn 基于qemu的模拟，所以必须给代码提供可运行的环境。最大的问题就是要关心内存处理。我们只想得到ollvm路径，而不是真实代码块的运行结果，因此要尽可能屏蔽非ollvm的内存操作。具体屏蔽方法稍后介绍。上面这段代码初始化Unicorn的虚拟CPU，并映射程序代码内存以及栈空间，最后调用hook_add设置UC_HOOK_CODE和UC_HOOK_MEM_UNMAPPED的事件回调。UC_HOOK_CODE回调会在每条指令执行前被调用，UC_HOOK_MEM_UNMAPPED会在内存异常的时候调用。

启动虚拟机

启动虚拟机的函数叫find_path，顾名思义，用于寻找真实块的下一个代码块。branch为分支控制。如果branch = 1，则虚拟机在遇到csel指令的时候会走csel条件分支，否则走csel条件相反的分支。至于如何控制虚拟机内部指令的执行，稍后介绍。

```
def find_path(start_addr,branch = None):
    global real_blocks
    global bin1
    global mu
    global list_trace
    global startaddr
    global distAddr
    global isSucess
    global branch_control
    try:
        list_trace = {}
        startaddr = start_addr
        isSucess = False
        distAddr = 0
        branch_control = branch
        mu.emu_start(start_addr,0x10000)
        print "emu end.."

    except UcError as e:
        pc = mu.reg_read(UC_ARM64_REG_PC)
        if pc != 0:
            #mu.reg_write(UC_ARM64_REG_PC, pc + 4)
            return find_path(pc + 4,branch)
        else:
            print("ERROR: %s  pc:%x" % (e,pc))
    if isSucess:
        return distAddr
    return None
```

控制流集成

使用队列的方式来路径搜索，起始搜索从函数入口开始。函数入口根据offset变量指定。queue中的元素是一个二元组，第一项为执行地址，第二项为寄存器环境。每次搜索开始的时候从queue中获取一个将要搜索的真实块，设置寄存器，调用find_path搜索下一个真实块，将搜索到的真实块与新寄存器放入队列（保证上下文完整）使用这样做的好处就是可以搜索任意队列中的代码块，并且寄存器环境一定是和该代码块一致的。

```
queue = [(offset,None)]
flow = {}
while len(queue) != 0:
    env = queue.pop()
    pc = env[0]
    set_context(env[1])
    item = list_blocks[pc]
    if pc in flow:
        #print "???"
        continue
    flow[pc] = []
    if item['ins'].find('csel') != -1:
        #raw_input()
        ctx = get_context()
        p1 = find_path(pc,0)
        if p1 != None:
            queue.append((p1,get_context()))
            flow[pc].append(p1)

        set_context(ctx)
        p2 = find_path(pc,1)

        if p1 == p2:
            p2 = None

        if p2 != None:
            queue.append((p2,get_context()))
            flow[pc].append(p2)
    else:
        p = find_path(pc)
        if p != None:
            queue.append((p,get_context()))
        flow[pc].append(p)
```

虚拟机的内部核心 hook_code

Unicorn在每一条汇编代码执行前会调用UC_HOOK_CODE回调，通过该回调可以控制虚拟机的一些行为。

这一节的代码均属于hook_code

跳过一条指令,直接修改PC寄存器的值为下一条指令的地址：

```
uc.reg_write(UC_ARM64_REG_PC, address + size)
```

路径探索，需要禁用掉一切函数调用、非栈空间内存访问，0x80000000为前面代码中提到的栈地址，当虚拟机指令有内存操作需求时，判断目标内存地址范围是否在栈中，如果不在栈中则跳过该指令。禁用的指令有bl、blx，只要识别bl前缀即可。

```
flag_pass = False
ban_ins = 'bl'
for b in ban_ins:
    if ins.mnemonic.find(b) != -1:
        flag_pass = True
        break

if ins.op_str.find('[') != -1:
    if ins.op_str.find('[sp') == -1:
        flag_pass = True
        for op in ins.operands:
            if op.type == ARM64_OP_MEM:
                addr = 0
                if op.value.mem.base != 0:
                    addr += mu.reg_read(reg_ctou(ins.reg_name(op.value.mem.base)))
                elif op.value.index != 0:
                    addr += mu.reg_read(reg_ctou(ins.reg_name(op.value.mem.index)))
                elif op.value.disp != 0:
                    addr += op.value.disp
                if addr >= 0x80000000 and addr < 0x80000000 +  0x10000 * 8:
                    flag_pass = False
if flag_pass:
    print("will pass 0x%x:\t%s\t%s" %(ins.address, ins.mnemonic, ins.op_str))
    uc.reg_write(UC_ARM64_REG_PC, address + size)
    return
```

寻找到下一个真实块的处理

```
if address in real_blocks and address != startaddr:
    isSucess = True
    distAddr = address
    print 'find:%x' % address
    uc.emu_stop()
    return
```

调试机制。
为了方便调试，我设计了一个简单的调试机制 在[ ]添加断点地址即可断点，！+ 寄存器即可查看寄存器的值。

```
if address in [  ] or debugflag:
    debugflag = True
    print("0x%x:\t%s\t%s" % (ins.address, ins.mnemonic, ins.op_str))
    while True:
        c = raw_input('>')
        if c == '':
            break
        if c == 's':
            uc.emu_stop()
            return
        if c == 'r':
            debugflag = False
            break
        if c[0] == '!':
            reg = reg_ctou(c[1:])
            print "%s=%x (%d)" % (c[1:], mu.reg_read(reg),mu.reg_read(reg))
            continue
```

含有branch的真实块处理。人工交换csel指令的值来实现分支。

```
if ins.mnemonic == 'csel':#ollvm branch
    print("csel 0x%x:\t%s\t%s" %(ins.address, ins.mnemonic, ins.op_str))
    regs = [reg_ctou(x) for x in ins.op_str.split(', ')]
    assert len(regs) == 4
    v1 = uc.reg_read(regs[1])
    v2 = uc.reg_read(regs[2])
    if branch_control == 1:
        uc.reg_write(regs[0],v1)
    else:
        uc.reg_write(regs[0],v2)
    uc.reg_write(UC_ARM64_REG_PC, address + size)
```

```
def hook_code(uc, address, size, user_data):
    global real_blocks
    global csel_addrs
    global list_trace
    global startaddr
    global debugflag
    global isSucess
    global distAddr
    global branch_control
    global list_blocks
    ban_ins = ["bl"]

    if isSucess:
        mu.emu_stop()
        #raw_input()
        return

    if address > end:
```

```
    for ins in md.disasm(bin1[address:address+size],address):
        print(">>> Tracing instruction at 0x%x, instruction size = 0x%x" % (address, size))
        print(">>> 0x%x:\t%s\t%s" % (ins.address, ins.mnemonic, ins.op_str))
        print

        if address in real_blocks:
            if list_trace.has_key(address):
                print "sssssss"
                ch = raw_input("This maybe a fake block. codesign:%s " % get_code_sign(list_blocks[address]))

                uc.emu_stop()
            else:
                list_trace[address] = 1

        if address in real_blocks and address != startaddr:
            isSucess = True
            distAddr = address
            print 'find:%x' % address
            # uc.reg_write(UC_ARM64_REG_PC,0xffffffff)
            uc.emu_stop()
            return

        flag_pass = False
        for b in ban_ins:
            if ins.mnemonic.find(b) != -1:
                flag_pass = True
                break

        if ins.op_str.find('[') != -1:
            if ins.op_str.find('[sp') == -1:
                flag_pass = True
                for op in ins.operands:
                    if op.type == ARM64_OP_MEM:
                        addr = 0
                        if op.value.mem.base != 0:
                            addr += mu.reg_read(reg_ctou(ins.reg_name(op.value.mem.base)))
                        elif op.value.index != 0:
                            addr += mu.reg_read(reg_ctou(ins.reg_name(op.value.mem.index)))
                        elif op.value.disp != 0:
                            addr += op.value.disp
                        if addr >= 0x80000000 and addr < 0x80000000 +  0x10000 * 8:
                            flag_pass = False
        if flag_pass:
            print("will pass 0x%x:\t%s\t%s" %(ins.address, ins.mnemonic, ins.op_str))
            uc.reg_write(UC_ARM64_REG_PC, address + size)
            return
        #breaks 0x31300
        if address in [ 0xB72EC ] or debugflag:
            debugflag = True
            print("0x%x:\t%s\t%s" % (ins.address, ins.mnemonic, ins.op_str))
            while True:
                c = raw_input('>')
                if c == '':
                    break
                if c == 's':
                    uc.emu_stop()
                    return
                if c == 'r':
                    debugflag = False
                    break
                if c[0] == '!':
                    reg = reg_ctou(c[1:])
                    print "%s=%x (%d)" % (c[1:], mu.reg_read(reg),mu.reg_read(reg))
                    continue

        if ins.mnemonic == 'ret':
            uc.reg_write(UC_ARM64_REG_PC, 0)
            isSucess = False
            print "ret ins.."
            mu.emu_stop()

        if ins.mnemonic == 'csel':#ollvm branch
            print("csel 0x%x:\t%s\t%s" %(ins.address, ins.mnemonic, ins.op_str))
            regs = [reg_ctou(x) for x in ins.op_str.split(', ')]
            assert len(regs) == 4
            v1 = uc.reg_read(regs[1])
            v2 = uc.reg_read(regs[2])
            if branch_control == 1:
                uc.reg_write(regs[0],v1)
            else:
                uc.reg_write(regs[0],v2)
            uc.reg_write(UC_ARM64_REG_PC, address + size)
```

绘制流程图

使用graphviz库绘制流程图。

```
def draw():
    queue = []
    dot = Digraph(comment='The Round Table')
    dot.attr('node', shape='box')
    queue.append(offset)
    check = {}
    while len(queue) > 0:
        pc = queue.pop()
        if check.has_key(pc):
            continue
        check[pc] = 1
        dot.node(hex(pc),list_blocks[pc]['ins'])
        for i in flow[pc]:
            if i != None:
                dot.edge(hex(pc), hex(i), constraint='true')
                queue.append(i)
    dot.render('test-output/round-table.gv', view=True)
```

比较完整的代码

```
from capstone import *
from capstone.arm64 import *
from graphviz import Digraph
from unicorn import *
from unicorn.arm64_const import *
from keystone import *



def reg_ctou(regname):#
    # This function covert capstone reg name to unicorn reg const.
    type1 = regname[0]
    if type1 == 'w' or type1 =='x':
        idx = int(regname[1:])
        if type1 == 'w':
            return idx + UC_ARM64_REG_W0
        else:
            if idx == 29:
                return  1
            elif idx == 30:
                return 2
            else:
                return idx + UC_ARM64_REG_X0
    elif regname=='sp':
        return 4
    return None

def asm_no_branch(ori,dist):
    ks = Ks(KS_ARCH_ARM64, KS_MODE_LITTLE_ENDIAN)
    print ("b #0x%x" % dist)
    ins, count  = ks.asm(("b #0x%x" % dist),ori)
    return  ins

def asm_has_branch(ori,dist1,dist2,cond):
    ks = Ks(KS_ARCH_ARM64, KS_MODE_LITTLE_ENDIAN)
    if ori == 0xb73b4:
        pass
    print "b%s #0x%x;b #0x%x" % (cond,dist1,dist2)
    ins, count = ks.asm("b%s #0x%x;b #0x%x" % (cond,dist1,dist2),ori)
    return ins

def hook_code(uc, address, size, user_data):
    global real_blocks
    global csel_addrs
    global list_trace
    global startaddr
    global debugflag
    global isSucess
    global distAddr
    global branch_control
    global list_blocks
    ban_ins = ["bl"]

    if isSucess:
        mu.emu_stop()
        #raw_input()
        return

    if address > end:
        uc.emu_stop()
        return

    for ins in md.disasm(bin1[address:address+size],address):
        print(">>> Tracing instruction at 0x%x, instruction size = 0x%x" % (address, size))
        print(">>> 0x%x:\t%s\t%s" % (ins.address, ins.mnemonic, ins.op_str))
        print

        if address in real_blocks:
            if list_trace.has_key(address):
                print "sssssss"
                ch = raw_input("This maybe a fake block. codesign:%s " % get_code_sign(list_blocks[address]))

                uc.emu_stop()
            else:
                list_trace[address] = 1

```

```
            print 'find:%x' % address
            uc.emu_stop()
            return

        flag_pass = False
        for b in ban_ins:
            if ins.mnemonic.find(b) != -1:
                flag_pass = True
                break

        if ins.op_str.find('[') != -1:
            if ins.op_str.find('[sp') == -1:
                flag_pass = True
                for op in ins.operands:
                    if op.type == ARM64_OP_MEM:
                        addr = 0
                        if op.value.mem.base != 0:
                            addr += mu.reg_read(reg_ctou(ins.reg_name(op.value.mem.base)))
                        elif op.value.index != 0:
                            addr += mu.reg_read(reg_ctou(ins.reg_name(op.value.mem.index)))
                        elif op.value.disp != 0:
                            addr += op.value.disp
                        if addr >= 0x80000000 and addr < 0x80000000 +  0x10000 * 8:
                            flag_pass = False
        if flag_pass:
            print("will pass 0x%x:\t%s\t%s" %(ins.address, ins.mnemonic, ins.op_str))
            uc.reg_write(UC_ARM64_REG_PC, address + size)
            return
        #breaks 0x31300
        if address in [ 0xB72EC ] or debugflag:
            debugflag = True
            print("0x%x:\t%s\t%s" % (ins.address, ins.mnemonic, ins.op_str))
            while True:
                c = raw_input('>')
                if c == '':
                    break
                if c == 's':
                    uc.emu_stop()
                    return
                if c == 'r':
                    debugflag = False
                    break
                if c[0] == '!':
                    reg = reg_ctou(c[1:])
                    print "%s=%x (%d)" % (c[1:], mu.reg_read(reg),mu.reg_read(reg))
                    continue

        if ins.mnemonic == 'ret':
            uc.reg_write(UC_ARM64_REG_PC, 0)
            isSucess = False
            print "ret ins.."
            mu.emu_stop()


        if ins.mnemonic == 'csel':#ollvm branch
            print("csel 0x%x:\t%s\t%s" %(ins.address, ins.mnemonic, ins.op_str))
            regs = [reg_ctou(x) for x in ins.op_str.split(', ')]
            assert len(regs) == 4
            v1 = uc.reg_read(regs[1])
            v2 = uc.reg_read(regs[2])
            if branch_control == 1:
                uc.reg_write(regs[0],v1)
            else:
                uc.reg_write(regs[0],v2)
            uc.reg_write(UC_ARM64_REG_PC, address + size)

def hook_mem_access(uc,type,address,size,value,userdata):
    pc = uc.reg_read(UC_ARM64_REG_PC)
    print 'pc:%x type:%d addr:%x size:%x' % (pc,type,address,size)
    #uc.emu_stop()
    return False


def get_code_sign(block):
    insn = block['capstone']
    sign = ''
    for i in insn:
        sign += i.mnemonic
        for op in i.operands:
            sign += str(op.type)
    return sign

def get_context():
    global mu
    regs = []
    for i in range(31):
        idx = UC_ARM64_REG_X0 + i
        regs.append(mu.reg_read(idx))
    regs.append(mu.reg_read(UC_ARM64_REG_SP))
    return regs

def set_context(regs):
    global mu
    if regs == None:
        return
    for i in range(31):
        idx = UC_ARM64_REG_X0 + i
        mu.reg_write(idx,regs[i])
    mu.reg_write(UC_ARM64_REG_SP,regs[31])
```

```
    global real_blocks
    global bin1
    global mu
    global list_trace
    global startaddr
    global distAddr
    global isSucess
    global branch_control
    try:
        list_trace = {}
        startaddr = start_addr
        isSucess = False
        distAddr = 0
        branch_control = branch
        mu.emu_start(start_addr,0x10000)
        print "emu end.."

    except UcError as e:
        pc = mu.reg_read(UC_ARM64_REG_PC)
        if pc != 0:
            #mu.reg_write(UC_ARM64_REG_PC, pc + 4)
            return find_path(pc + 4,branch)
        else:
            print("ERROR: %s  pc:%x" % (e,pc))
    if isSucess:
        return distAddr
    return None

ssign = {u'b2',
 u'cmp11b.ne2',
 u'cmp11mov11b.ne2',
 u'movz12movk12b2',
 u'movz12movk12cmp11b.ne2',
 u'movz12movk12cmp11mov11b.ne2',
 'movz12movk12cmp11mov11movz12movk12movz12movk12b.ne2',
 u'movz12movk12cmp11movz12movk12b.eq2',
 'movz12movk12cmp11movz12movk12b.ne2',
 'movz12movk12movz12movk12b2',
 'movz12movk12cmp11movz12movk12movz12movk12movz12movk12b.eq2',
 'movz12movk12movz12movk12cmp11b.eq2',
 'movz12movk12movz12movk12cmp11movz12movk12b.eq2',
 'movz12movk12cmp11b.eq2',
         'ldr13b2',
         'mov11movz12movk12cmp11movz12movk12b.eq2'
 }
ssign2 = set()
def  is_real_blocks(ins):
    sign = get_code_sign(ins)
    if sign in ssign:
        return False
    if sign.endswith('movk12movz12movk12b.ne2'):
        return False

    for insn in item['capstone']:
        #print insn.mnemonic
        if insn.mnemonic not in ['movz','movk','cmp','b.eq','b.ne']:
            return True
    ssign2.add(sign)
    return False

def draw():
    queue = []
    dot = Digraph(comment='The Round Table')
    dot.attr('node', shape='box')
    queue.append(offset)
    check = {}
    while len(queue) > 0:
        pc = queue.pop()
        if check.has_key(pc):
            continue
        check[pc] = 1
        dot.node(hex(pc),list_blocks[pc]['ins'])
        for i in flow[pc]:
            if i != None:
                dot.edge(hex(pc), hex(i), constraint='true')
                queue.append(i)
    dot.render('test-output/round-table.gv', view=True)


def print_real_blocks():
    print '######################### real blocks ##########################'
    cnt = 0
    for i in real_blocks:
        item = list_blocks[i]
        print item['ins']
        print
        if cnt == 50:
            c = raw_input()
            if c == 'c':
                return
        cnt += 1


offset = 0x70438 #function start
end = 0x7170C    #function end

bin1 = open('libvdog.so','rb').read()
md = Cs(CS_ARCH_ARM64,CS_MODE_ARM)
md.detail = True #enable detail analvise
```

```
block_item = {}
processors = {}
dead_loop = []
real_blocks = []
csel_list_cond = {}


isNew = True
insStr = ''
for i in md.disasm(bin1[offset:end],offset):
    insStr += "0x%x:\t%s\t%s\n" %(i.address, i.mnemonic, i.op_str)
    if isNew:
        isNew = False
        block_item = {}
        block_item["saddress"] = i.address
        block_item['capstone'] = []

    block_item['capstone'].append(i)
    block_item['flag'] = False


    if len(i.groups) > 0 or i.mnemonic == 'ret':
        isNew = True
        block_item["eaddress"] = i.address
        block_item['ins'] = insStr
        insStr = ''
        for op in i.operands:
            if op.type == ARM64_OP_IMM:
                block_item["naddress"] = op.value.imm

                if op.value.imm == i.address:
                    print "dead loop:%x" % i.address
                    dead_loop.append(i.address)

                if not processors.has_key(op.value.imm):
                    processors[op.value.imm] = 1
                else:
                    processors[op.value.imm] += 1
        if not block_item.has_key("naddress"):
            block_item['naddress'] = None
        list_blocks[block_item["saddress"]] = block_item

#delete dead loop
for dead in dead_loop:
    if processors.has_key(dead):
        del(processors[dead])

for b in list_blocks:
    if processors.has_key(list_blocks[b]['naddress']):
        if processors[list_blocks[b]['naddress']] > 1:
            real_blocks.append(list_blocks[b]['saddress'])

fake_blocks = []
for i in real_blocks:
    item = list_blocks[i]
    if not is_real_blocks(item):
        print '## fake block ###'
        print item['ins']
        fake_blocks.append(i)

for x in fake_blocks:
    real_blocks.remove(x)

for i in list_blocks:
    if list_blocks[i]['ins'].find('ret') != -1:
        print 'ret block:%x' % i
        real_blocks.append(i)

print '######################## real blocks ##########################'
print [hex(x) for x in real_blocks]

mu = Uc(UC_ARCH_ARM64, UC_MODE_ARM)
#init stack
mu.mem_map(0x80000000,0x10000 * 8)

mu.mem_map(0, 4 * 1024 * 1024)
mu.mem_write(0,bin1)
mu.reg_write(UC_ARM64_REG_SP, 0x80000000 + 0x10000 * 6)
mu.hook_add(UC_HOOK_CODE, hook_code)
mu.hook_add(UC_HOOK_MEM_UNMAPPED,hook_mem_access)

list_trace = {}
debugflag = False



if offset in real_blocks:
    real_blocks.remove(offset)

queue = [(offset,None)]
flow = {}
while len(queue) != 0:
    env = queue.pop()
    pc = env[0]
    set_context(env[1])
    item = list_blocks[pc]
    if pc in flow:
        #print "???"
        continue
```

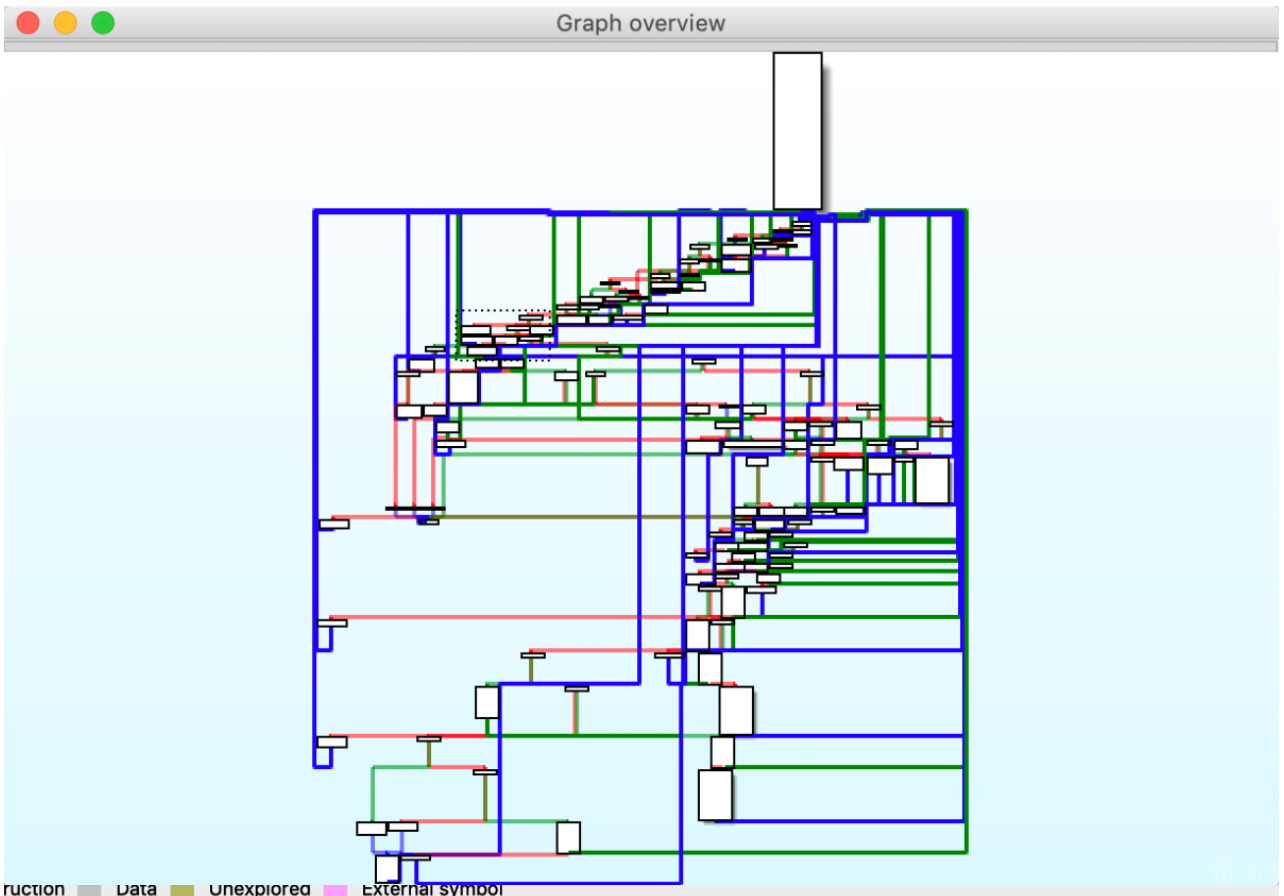
```
        #raw_input()
        ctx = get_context()
        p1 = find_path(pc,0)
        if p1 != None:
            queue.append((p1,get_context()))
            flow[pc].append(p1)

        set_context(ctx)
        p2 = find_path(pc,1)

        if p1 == p2:
            p2 = None

        if p2 != None:
            queue.append((p2,get_context()))
            flow[pc].append(p2)
    else:
        p = find_path(pc)
        if p != None:
            queue.append((p,get_context()))
        flow[pc].append(p)
draw()
```

一些效果图



IDA View-A | IDA View-B | IDA View-C | Pseudocode-A | Hex View-1

```
77          {
78            while ( 1 )
79            {
80              while ( 1 )
81              {
82                while ( 1 )
83                {
84                  while ( 1 )
85                  {
86                    v8 = v7;
87                    if ( v7 > -1796611739 )
88                      break;
89                    if ( v7 == -1974907953 )
90                      v7 = -1591680163;
91                  }
92                  if ( v7 <= 2122517887 )
93                    break;
94                  if ( v7 == 2122517888 )
95                  {
96                    a1 = sub_D030();
97                    v7 = 484421971;
98                  }
99                }
100               if ( v7 <= 2071858715 )
101                 break;
102               if ( v7 == 2071858716 )
103               {
104                 a1 = sub_D940(v44, "sg.bigo.enterprise.live:service");
105                 v54 = a1;
106                 v7 = -1796611738;
107               }
108             }
109             if ( v7 <= 2038211856 )
110               break;
111             if ( v7 == 2038211857 )
112             {
113               a1 = sub_D030();
114               v7 = -208592878;
115             }
```

修复后

```
45  if ( (_DWORD)a30 != 2 && (a33 = a34, sub_76F60(a33) & 1) )
46  {
47LABEL_58:
48    HIDWORD(a12) = -1;
49  }
50  else
51  {
52    a25 = a14;
53    v78 = sub_DCA0(a14, 0LL, 2000LL);
54    a10 = a14;
55    v79 = sub_CFA0(v78);
56    sub_D690(a10, "/proc/%d/cmdline", v79);
57    v72 = sub_DCB0(a10, &unk_C5E4D);
58    a26 = v72;
59    if ( !a26 )
60    {
61      a22 = a15;
62      sub_DCA0(a15, 0LL, 2000LL);
63      a23 = a15;
64      sub_DDF0(a26, "%s", a15);
65      sub_D1F0(a26);
66      if ( sub_D530(a23, 58LL) || (v72 = sub_D940(a23, "sg.bigo.enterprise.live:ser
67        v72 = sub_D330();
68    }
69    HIDWORD(a24) = *off_DFF18;
70    if ( HIDWORD(a24) != 15 )
71      v72 = sub_D2A0();
72    sub_DD60(v72);
73    if ( sub_CB00(a34) & 1 )
74      sub_D030();
75    v74 = (_BYTE *)sub_2E998();
76    a21 = v74;
77    if ( !*v74 )
78    {
79      sub_D0A0();
80      a18 = *(_QWORD *)a44;
81      HIDWORD(a19) = sub_D5A0(a18, a21);
82      if ( !HIDWORD(a19) )
83        sub_D030();
0007100C JNI_OnLoad:44 (7100C)
```

最后

还原patch的代码已经被我阉割，因为不是很完善，我仅提供一个非符号执行的思路供大家研究。

完整代码见附件，代码反混淆的目标是JNI_OnLoad，可以自己修改offset和end来修改目标。输出文件为bin.out。

今年高考没考好，过几天就去复读了，没有太多时间搞这个了。

欢迎讨论 + 提供ARM64的样本。

[公告]请完善个人简历信息，招聘企业等你来！

最后于 2019-6-29 16:20 被无名侠编辑，原因:

上传的附件:

MobileFile.zip（632.00kb，396次下载）

收藏 · 85 点赞 · 13 打赏 分享

最新回复 (35)

yikuaiyingbi 2019-6-29 16:37 2 楼 0

膜。

极客

首页 论坛 课程 招聘 发现

最新回复 (35)

卓桐 4 2019-6-29 16:51 3 楼 0

666

专家

上海刘一刀 2 2019-6-29 17:38 4 楼 0

6666666666666666666666666

大牛

huluxia 2019-6-29 17:41 5 楼 0

顶，加油

极客

大帅锅 3 2019-6-29 18:00 6 楼 1

侠哥还是这么牛，自从看了侠哥的视频（尤其是下楼给我买冰糕的那集）从此妈妈再也不用担心我的学习了。

专家

无名侠 10 2019-6-29 20:41 7 楼 0

> 大帅锅 侠哥还是这么牛，自从看了侠哥的视频（尤其是下楼给我买冰糕的那集）从此妈妈再也不用担心我的学习了。

什么鬼东西？？？？

学者

Vn小帆 2019-6-30 01:13 8 楼 0

收藏了

极客

qj111111 2019-6-30 12:11 9 楼 0

谢谢楼主

极客

smartdon 1 2019-6-30 20:17 10 楼 0

正想研究下，谢谢楼主了

大侠

葫芦娃 1 2019-6-30 20:25 11 楼 1

> 无名侠 什么鬼东西？？？？

雪糕给你买好了，哥哥

大牛

crackJ 1 2019-6-30 21:35 12 楼 0

这混淆的代码貌似没加垃圾指令与无用分支，ollvm一共三种混淆方式，互相配合，再加大强度，其实还是很难还原的

大侠

Dstlemoner 2019-7-1 01:31 13 楼 0

为什么要复读。。。。。专升本都可以。。。。复读浪费时间。。

极客

苋杉 2019-7-1 09:42 14 楼 0

66666666666666666666

极客

roysue 3 2019-7-1 14:41 15 楼 0

> 无名侠 什么鬼东西？？？？

雪糕给你买好了，哥哥

版主

最新回复 (35)

gtict 4 2019-7-2 19:06 16 楼 0

极客

cmp11b.ne2 这个指令怎么来的。ida没看到带11的

双子座_461389 2 2019-7-2 21:53 17 楼 0

极客

谢谢 楼主啦

mb_mnvljacx 0 2019-7-3 14:26 18 楼 0

临时

招聘一个逆向工程师13996084443同微信

goodgirls 2 2019-7-3 21:59 19 楼 0

极客

看到雪糕，好好笑

supperlitt 6 2019-7-15 08:42 20 楼 0

极客

膜拜

蔡金成 2 2019-7-25 10:03 21 楼 0

极客

是个神仙。

fcguo 2 2019-7-30 17:42 22 楼 0

极客

大神好啊，正好有个娜迦的APP,以前打开过这个软件的老版本，无壳。今天更新，发现无法用反编译软件Android killer打开，apk ide也报错。用Jadx-gui打开后，发现根目录下的dex体积很小，比较异常。后来在/asset/main000下发现三个dex文件，若干so文件，其中有libhdog.so、libvdog.so，如何脱壳修复啊，期待大神的教程，非常感谢，地址：https://mobile.e-chinalife.com/ChinalifeRenew/APK/ehome_mobile_for_plus_last_61.apk

AqCxBoM 2 2019-8-4 18:26 23 楼 0

极客

感谢分享 拜读

Vn小帆 5 2019-9-20 18:55 24 楼 0

极客

好文章

kanxue666 0 2019-10-20 09:56 25 楼 0

临时

年轻有为啊，继续努力

大C滑稽 2 2019-10-20 19:23 26 楼 0

极客

> 无名侠 什么鬼东西？？？？

请问大佬还开培训吗？想学

liumengde 3 2019-10-23 11:28 27 楼 0

极客

arm反混淆貌似解决不了 它没有csel分支指令

wuaiwu 2 3 2019-11-19 15:07 28 楼 0

专家

以后工作要有个本科学历当敲门砖的，对你将来职业发展有好处。小伙子很不错啊，希望你继续努力。（特意登录多年不上的账号来给你留言）

最新回复 (35)

artake 2 2019-12-10 09:48 29 楼 0

大侠

学历还是很重要的，加油!

lookzo 3 2019-12-10 09:55 30 楼 0

极客

不错，先支持一下

adime_9001 0 2019-12-10 16:10 31 楼 0

临时

谢谢楼主分享

小黄鸭爱学习 4 2019-12-13 18:22 32 楼 0

极客

> Dstlemoner 为什么要复读。。。。。专升本都可以。。。。复读浪费时间。。

复读如果可以提升一个阶段，那么可以说是从长远看，收益最大的事情了 。
在第一学历歧视的大环境下， 专升本的 收益远小于复读。

最后于 2019-12-13 18:23 被小黄鸭爱学习编辑，原因:

挤蹭菌衣 4 2020-3-6 16:36 33 楼 0

大侠

膜拜大佬，这玩意好难，现在卡在这了

jerryxjtu 2 6 2020-4-11 17:53 34 楼 0

专家

谢谢分享。
对使用capstone做basic block划分不理解。
i.groups的意思是这个指令在语义上属于哪个分组。
https://www.capstone-engine.org/lang_python.html
Check if this instruction belongs to any semantic group. If so, print out all group ID.
例如 bl #0x12b5b8
i.groups返回1和7，从代码上看好像对应:
ARM64_GRP_JUMP 1
ARM64_GRP_BRANCH_RELATIVE 7

用capstone-4.0.1跑附件的so，看到的分组是这样:
start 0x70438L, end 0x704c8L
start 0x70438L, end 0x704c8L
start 0x70438L, end 0x704c8L
start 0x70438L, end 0x704c8L
start 0x70438L, end 0x704c8L
start 0x70438L, end 0x704c8L
start 0x70438L, end 0x704c8L
start 0x70438L, end 0x704c8L
怎么能获得正确的分组呢?

jerryxjtu 2 6 2020-4-11 17:56 35 楼 0

专家

实际操作中发现，用代码很难界定什么样的块是OB块，
如果误删了正常块，就会连锁导致一大堆的逻辑丢失。

无名侠 5 10 2020-5-23 22:42 36 楼 0

学者

> jerryxjtu 实际操作中发现，用代码很难界定什么样的块是OB块， 如果误删了正常块，就会连锁导致一大堆的逻辑丢失。

我已经解决这个问题，等过段时间公开

最新回复 (35)

游客

登录 | 注册 方可回帖

回帖 表情 高级回复

返回

©2000-2020 看雪学院 | Based on Xiuno BBS
域名：加速乐 | SSL证书：亚洲诚信 | 安全网易易盾| 同盾反欺诈| 服务器：绿盟科技
看雪APP | 公众号：ikanxue | 关于我们 | 联系我们 | 企业服务
Processed: **0.448**s, SQL: **85** / 京ICP备10040895号-17